

Musterlösung Praktikum 5: B-Baum

1 Graphviz-Visualisierung

1.1 Implementierung von BTreeVis

BTreeVis erbt von BTree und fügt eine Methode `graph.traversal()` hinzu, die den gesamten Baum rekursiv durchläuft und als Graphviz-DOT-Datei exportiert:

```
class BTreeVis(BTree):

    def graph_traversal(self):
        def _node_rep(node):
            label = "└─┬─".join(str(node.value[i]) for i in range(node.n)
                                )
            dot.node(str(id(node)), label=label,
                    shape="record", fontname="Arial")

        def _rec(node):
            _node_rep(node)
            if not node.leaf:
                for i in range(node.n + 1):
                    child = node.children[i]
                    if child is not None:
                        dot.edge(str(id(node)), str(id(child)))
                        _rec(child)

        dot = graphviz.Digraph(
            name=f"BTree_m{self.m}", engine="dot",
            node_attr={"fontname": "Arial"}, format="pdf")
        _rec(self.root)
        ts = datetime.now().strftime("%Y%m%d_%H%M%S")
        dot.render(f"BTree_m{self.m}_{ts}.gv")
```

Jeder Knoten wird als Record-Knoten dargestellt – der Label enthält alle im Knoten gespeicherten Schlüssel, getrennt durch `|`. Kanten verbinden einen Knoten mit jedem seiner $n + 1$ Kindzeiger; Kantenrichtungen werden nur durch die Eltern-Kind-Beziehung bestimmt und nicht durch die Position eines bestimmten Kindknotens.

Der Aufruf für `seq1.txt` der Ordnung 3 sieht wie folgt aus:

```
ctx = AlgoContext()
values = Array.from_file('data/seq1.txt', ctx)
tree = BTreeVis(3, ctx)
for cell in values:
    tree.insert(cell)
tree.graph_traversal() # erzeugt BTree_m3_<ts>.gv und .pdf
print("Schlüssel└─┬─in└─┬─der└─┬─Wurzel:", tree.root.n)
print("Hoehe└─┬─des└─┬─Baums:", tree.height())
```

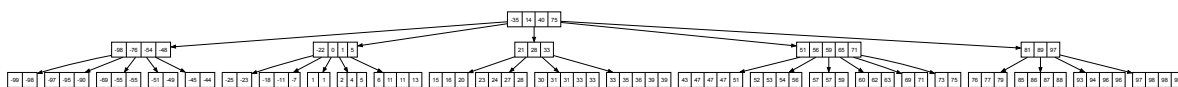


Abbildung 1: B-Baum der Ordnung 3 nach Einfügen von seq1.txt

1.2 Beobachtungen am erzeugten Baum

Wie viele Schlüssel enthält die Wurzel?

Nach dem Befüllen mit seq1.txt enthält die Wurzel **4 Schlüssel** (`tree.root.n == 4`). Das liegt im erlaubten Bereich $[m - 1, 2m - 1] = [2, 5]$ für Ordnung 3; nur die Wurzel darf auch weniger als $m - 1$ Schlüssel haben.

Liegen alle Blattknoten auf derselben Tiefe? Warum ist das beim B-Baum immer so?

Ja – alle Blattknoten liegen stets auf derselben Tiefe. Das ist eine fundamentale Invariante des B-Baums:

- *Splits* propagieren ausschließlich *aufwärts*: Wenn ein Knoten voll ist, wird sein mittlerer Schlüssel in den Elternknoten verschoben und der Knoten in zwei neue Knoten aufgeteilt. Die entstehenden Knoten liegen auf derselben Ebene wie der ursprüngliche.
- Die *Wurzel* wächst nach oben: Nur wenn die Wurzel selbst gesplittet wird, entsteht eine neue Wurzel auf einer höheren Ebene – damit erhöht sich die Höhe des gesamten Baums gleichzeitig für *alle* Pfade um 1.
- *Merges* beim Löschen wirken symmetrisch: Auch beim Schrumpfen sinkt die Höhe immer global (wenn die Wurzel leer wird), nie nur auf einzelnen Pfaden.

Da die Baumhöhe nur durch globale Ereignisse (Splitten oder Mergen der Wurzel) verändert wird, haben alle Blätter zu jedem Zeitpunkt exakt dieselbe Tiefe.

2 Eigenschaften des B-Baums

2.1 Höhe für Ordnung 3 und Ordnung 5

```
ctx = AlgoContext()
values = Array.from_file('data/seq3.txt', ctx)
tree3 = BTree(3, ctx)
tree5 = BTree(5, ctx)
for cell in values:
    tree3.insert(cell)
    tree5.insert(cell)
print("Höhe_Ordnung_3:", tree3.height())    # 6
print("Höhe_Ordnung_5:", tree5.height())    # 4
```

seq3.txt enthält 10000 Werte. Die theoretischen Höhengrenzen lauten:

$$h \leq \log_m \left(\frac{n+1}{2} \right) \implies \text{Ordnung 3: } h \leq 8,3 \quad \text{Ordnung 5: } h \leq 5,7$$

Die gemessenen Werte (6 bzw. 4) liegen deutlich darunter, weil die Knoten im Durchschnitt m bis $2m - 1$ Schlüssel enthalten – also mehr als das erlaubte Minimum.

2.2 Traversierung und Sortierungsprüfung

```
results3 = []
tree3.traversal(lambda v: results3.append(int(str(v))))
print("Sortiert_␣(Ordnung_␣3):", results3 == sorted(results3))    # True

results5 = []
tree5.traversal(lambda v: results5.append(int(str(v))))
print("Sortiert_␣(Ordnung_␣5):", results5 == sorted(results5))    # True
```

Die In-Order-Traversal eines B-Baums liefert stets eine aufsteigend sortierte Folge. Das folgt direkt aus der B-Baum-Eigenschaft: In jedem Knoten gilt $c_0 < k_0 < c_1 < k_1 < \dots < k_{n-1} < c_n$, wobei k_i die gespeicherten Schlüssel und c_i die Teilbäume (Kindzeiger) sind. `traversal` besucht rekursiv $c_0, k_0, c_1, k_1, \dots$, also in aufsteigender Reihenfolge der Werte.

2.3 Knoten mit dem Wert 0

```
node3 = tree3.search(0)
node5 = tree5.search(0)
print("Knoten_␣mit_␣0_␣(Ordnung_␣3):", node3)    # (0 2 5)
print("Knoten_␣mit_␣0_␣(Ordnung_␣5):", node5)    # (-14 -10 -5 0)
```

Warum enthält der Knoten bei Ordnung 5 tendenziell mehr Schlüssel als bei Ordnung 3?

Die Mindest- und Höchstzahl von Schlüsseln pro Knoten hängt von m ab:

	Ordnung 3	Ordnung 5
Minimum (Nicht-Wurzel)	$m - 1 = 2$	$m - 1 = 4$
Maximum	$2m - 1 = 5$	$2m - 1 = 9$

Bei höherer Ordnung fasst jeder Knoten mehr Schlüssel, bevor ein Split ausgelöst wird. Im Gleichgewicht enthält ein typischer Knoten ungefähr $\frac{3m}{2}$ Schlüssel (Mitte zwischen Minimum und Maximum). Da dieser Erwartungswert mit m wächst, liegen bei Ordnung 5 im Schnitt mehr Schlüssel in einem Knoten – und damit auch mehr Nachbarschlüssel neben dem gesuchten Wert 0.

3 Disk-I/O und der Parameter m

3.1 Messung

```
N = 500
orders = [2, 3, 5, 10, 20]
ctx = AlgoContext()
base = Array.random(N, -1000, 1000, ctx)

for m in orders:
    ctx.reset()
    tree = BTree(m, ctx)
    for cell in base:
        tree.insert(cell)
    stats[m] = {
        "comparisons": ctx.comparisons,
        "loads":       count_loads(tree.root),
        "saves":       count_saves(tree.root),
        "height":      tree.height(),
    }
```

Repräsentative Messergebnisse für $n = 500$ zufällige Werte:

m	Vergleiche	Disk-Loads	Disk-Saves	Höhe
2	3 915	3 238	1 316	6
3	4 211	2 163	962	4
5	5 068	1 531	746	3
10	7 179	1 241	611	2
20	9 903	979	554	1

3.2 Analyse der Messungen

Warum sinkt die Zahl der Plattenzugriffe mit wachsendem m ?

Größeres m bedeutet mehr Schlüssel pro Knoten und damit weniger Knoten insgesamt – die Baumhöhe $h \approx \log_m n$ sinkt. Jede Einfügeoperation durchläuft einen Pfad der Länge h ; an jedem Knoten auf diesem Pfad wird ein `load()` und ggf. ein `save()` ausgelöst. Weniger Ebenen $\Rightarrow$ weniger Disk-Zugriffe.

Warum steigen die internen Vergleiche mit wachsendem m , obwohl die Baumhöhe sinkt?

Innerhalb eines Knotens mit bis zu $2m - 1$ Schlüsseln werden die Schlüssel *linear* durchsucht: im schlechtesten Fall $2m - 1$ Vergleiche pro Knoten. Die Gesamtkosten pro Einfügung betragen deshalb:

$$\underbrace{\mathcal{O}(m)}_{\text{Vergleiche/Knoten}} \cdot \underbrace{\mathcal{O}(\log_m n)}_{\text{Höhe}} = \mathcal{O}(m) \cdot \mathcal{O}\left(\frac{\log n}{\log m}\right) = \mathcal{O}\left(\frac{m}{\log m} \cdot \log n\right)$$

Der zweite Schritt verwendet den Basiswechsel $\log_m n = \frac{\log n}{\log m}$ (mit einem beliebigen, festen Logarithmus, z. B. $\log_2$). Da $m/\log m$ streng monoton wächst, steigen die Vergleiche kontinuierlich mit m – unabhängig von der geringeren Höhe. Der B-Baum spart also Disk-I/O auf Kosten von mehr CPU-Vergleichen.

Welche Ordnung m ist optimal für 4-KB-Blöcke?

Ein Knoten der Ordnung m enthält:

- $2m - 1$ Schlüssel (8-Byte-Integer),
- $2m$ Kindzeiger (Blocknummern, ebenfalls 8 Byte).

Die Knotengröße in Byte beträgt:

$$(2m - 1) \cdot 8 + 2m \cdot 8 = 8(4m - 1)$$

Gleichsetzen mit der Blockgröße:

$$8(4m - 1) = 4096 \implies 4m - 1 = 512 \implies m = \frac{513}{4} \approx 128$$

Die optimale Ordnung ist $m = 128$. Disk-I/O-Kosten dominieren gegenüber CPU-Vergleichen, daher ist ein großes m sinnvoll – bei $m = 128$ passt jeder Knoten exakt in einen Disk-Block, und die Baumhöhe beträgt für $n = 10^6$ Einträge nur $\lceil \log_{128}(10^6) \rceil = 3$ Ebenen.